Guitare électrique : Étude de l'influence du type de bobinage dans le micro

TIPE n° 39662

CPGE PT

Session 2026

Annexe : Code python

Table des matières

1 Bibliothèque de la modélisation **2**
1.1 Visualisation des données . . . 2
1.1.1 Enregistrer un graphique . . . 2
1.1.2 Calculer une FFT . . . 3
1.2 Modélisation électrique & Optimisation numérique . . . 3
1.2.1 Impédance théorique d'un micro . . . 3
1.2.2 Calcul de la matrice Jacobienne . . . 4
1.2.3 Calcul des écarts . . . 4
1.2.4 Algorithme des moindres carrés . . . 5
1.2.5 Calcul du diagramme de Bode d'un micro chargé . . . 7
1.3 Mise en évidence du filtrage spatial . . . 8

2 Traitement des données expérimentales **9**
2.1 Filtrage des données (Passe-bas numérique) . . . 9
2.2 Amplitude d'un signal périodique . . . 9

3 Données et valeurs numériques **10**
3.1 Caractéristiques électriques et mécaniques . . . 10
3.2 Caractéristiques magnétiques . . . 10

1 Bibliothèque de la modélisation

1.1 Visualisation des données

1.1.1 Enregistrer un graphique

Cette fonction automatise la génération de graphiques standardisés `matplotlib`, en intégrant notamment le surlignage des zones fréquentielles caractéristiques de la guitare.

```python
import matplotlib.pyplot as plt
import numpy as np

def save_plot(fig_path: str, plot_calls, title: str = "", xlabel: str = "", ylabel: str = "", ylim = None, xlim = None, legend: bool = True, log: bool = False, vspan: bool = False):
    """Enregistre une figure matplotlib avec des options de personnalisation.
    Args:
        fig_path (str): Chemin vers le fichier de sauvegarde
        plot_calls (tuple): Appels de tracé sous la forme (x, y, kwargs)
        title (str): Titre de la figure
        xlabel (str): nom de l'axe des x
        ylabel (str): nom de l'axe des y
        xlim (tuple): Limites de l'axe des x
        ylim (tuple): Limites de l'axe des y
        legend (bool): presence de la legende
        log (bool): presence d'une echelle logarithmique sur l'axe des x
        vspan (bool): presence des zones de frequences fondamentales et harmoniques
        error_bars (bool): active l'utilisation de plt.errorbar (necessite yerr/xerr dans kwargs)
    """
    plt.figure(figsize=(6, 5))
    handles_curves = []
    z_handles = []

    if vspan:
        # Zone des fondamentales de la guitare (Mi2 au Mi6)
        z1 = plt.axvspan(82.41, 1174.66, color="#F0E68C", alpha=0.3, label="Fréquences fondamentales", zorder=0)
        # Zone des harmoniques supérieures
        z2 = plt.axvspan(1174.66, 5873.3, facecolor="none", alpha=0.5, hatch="////", edgecolor="#424242", label="Fréquences harmoniques", zorder=0)
        z_handles = [z1, z2]

    for call in plot_calls:
        x, y, kwargs = call
        line, = plt.plot(x, y, **kwargs)
        handles_curves.append(line)

    if title: plt.title(title)
    if xlabel: plt.xlabel(xlabel)
    if ylabel: plt.ylabel(ylabel)
    if ylim is not None: plt.ylim(ylim)
    if xlim is not None: plt.xlim(xlim)

    plt.grid(True, alpha=0.6)
    if log: plt.xscale('log')

    if legend:
        leg_courbes = plt.legend(handles=handles_curves, loc='best', framealpha=1)
        if vspan:
            plt.gca().add_artist(leg_courbes)
            plt.legend(handles=z_handles, title="Zones d'étude", bbox_to_anchor=(0.5, -0.18), loc='upper center', ncol=2, borderaxespad=0.)

    plt.tight_layout()
    plt.savefig(fig_path, dpi=300)
    plt.close()
```

1.1.2 Calculer une FFT

```python
import numpy as np

def compute_and_normalize_fft(signal: np.ndarray, Ne_fft: int):
    """Calcule et normalise l'amplitude du spectre de Fourier d'un signal."""
    fft = np.fft.fft(signal)
    amp = np.abs(fft) / Ne_fft
    return amp
```

1.2 Modélisation électrique & Optimisation numérique

1.2.1 Impédance théorique d'un micro

```python
import numpy as np

def Z_micro(X: np.ndarray, rs: float, w: np.ndarray) -> np.ndarray:
    """Calcul l'impedance d'un micro de guitare en une pulsation w"""
    L, C_n, Rp_k = X
    C, Rp = C_n * 1e-9, Rp_k * 1e3  # Conversion en F et en Ohm

    # Impédance du bloc parallèle (Rp // C)
    den_para = 1 + (Rp * C * w)**2
    Z_para_re = Rp / den_para
    Z_para_im = - (Rp**2 * C * w) / den_para

    # Impédance totale = Z_serie + Z_parallele
    Z_re = rs + Z_para_re
    Z_im = (L * w) + Z_para_im

    return np.sqrt(Z_re**2 + Z_im**2)
```

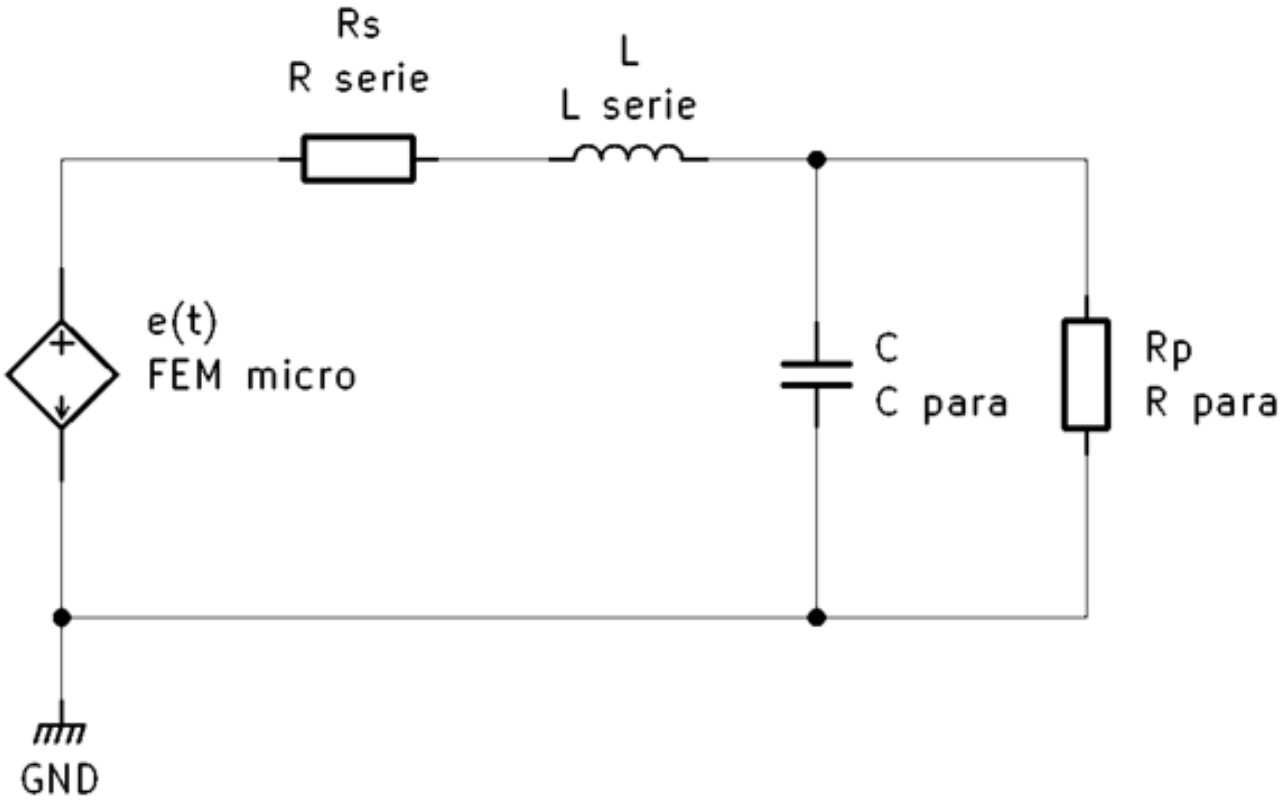


FIGURE 1 – Schéma électrique équivalent des micros

1.2.2 Calcul de la matrice Jacobienne

La matrice jacobienne rassemble les dérivées partielles de chaque composante du vecteur des résidus par rapport aux trois paramètres à optimiser (L, C, R_p). Elle fournit une approximation linéaire locale de la sensibilité du modèle, ce qui permet à l'algorithme de Levenberg-Marquardt de déterminer la direction et l'amplitude de la correction optimale à chaque itération.

```python
import numpy as np

def Matrice_Jacobienne(X:np.ndarray, rs:float, f:np.ndarray)->np.ndarray:
    """Basee sur l'algorythme de Levenberg_Marquardt, cette fonction retourne la matrice Jacobienne J
    Args:
        X (np.ndarray): Vecteur des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
        Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
        rs (float):Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
    Returns:
        np.ndarray: Matrice Jacobienne definie tel que chaque colonne i contient les derivees partielles par rapport a la variable situe en i dans X pour toute
        pulsation w
    """
    w = 2 * np.pi * f
    L, C_n, Rp_k = X
    C, Rp = C_n * 1e-9, Rp_k * 1e3

    # Variables intermediaires
    den_L = rs**2 + (L * w)**2
    Y_re = (1 / Rp) + (rs / den_L)
    Y_im = (C * w) - (L * w / den_L)
    Z = 1 / np.sqrt(Y_re**2 + Y_im**2)
    Z3 = Z**3

    # Derivees partielles de Y
    dYre_dL = - (rs * 2 * L * w**2) / (den_L**2)
    dYim_dL = - (w * (rs**2 - (L * w)**2)) / (den_L**2)
    dZ_dL = - Z3 * (Y_re * dYre_dL + Y_im * dYim_dL)

    dYim_dCn = w * 1e-9
    dZ_dCn = - Z3 * (Y_im * dYim_dCn)

    dYre_dRpn = - 1 / (1000 * Rp_k**2)
    dZ_dRpk = - Z3 * (Y_re * dYre_dRpn)

    return np.column_stack((dZ_dL, dZ_dCn, dZ_dRpk))
```

1.2.3 Calcul des écarts

Le vecteur des résidus quantifie l'écart entre le module de l'impédance expérimentale et celui calculé par le modèle théorique pour chaque point de fréquence mesuré. Il constitue l'erreur brute globale que l'algorithme des moindres carrés cherche à minimiser en ajustant les paramètres électriques du micro.

```python
import numpy as np

def residus(X:np.ndarray, rs:float, f:np.ndarray, Z_exp:np.ndarray)-> np.ndarray:
    """Calcul l'ecart (l'erreur) entre l'impedance calcule theoriquement et l'impedance reelle experimentale
    Args:
        X (np.ndarray): Vecteur des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
        Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
        rs (float):Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
        Z_exp (np.ndarray): Tableau Numpy des impedances experimentale releve en Ohm
    Returns:
        np.ndarray: Tableau Numpy des distances separant chaque point theorique au point experimentaux
    """
    return Z_exp - Z_micro(X, rs, 2 * np.pi * f)
```

1.2.4 Algorithme des moindres carrés

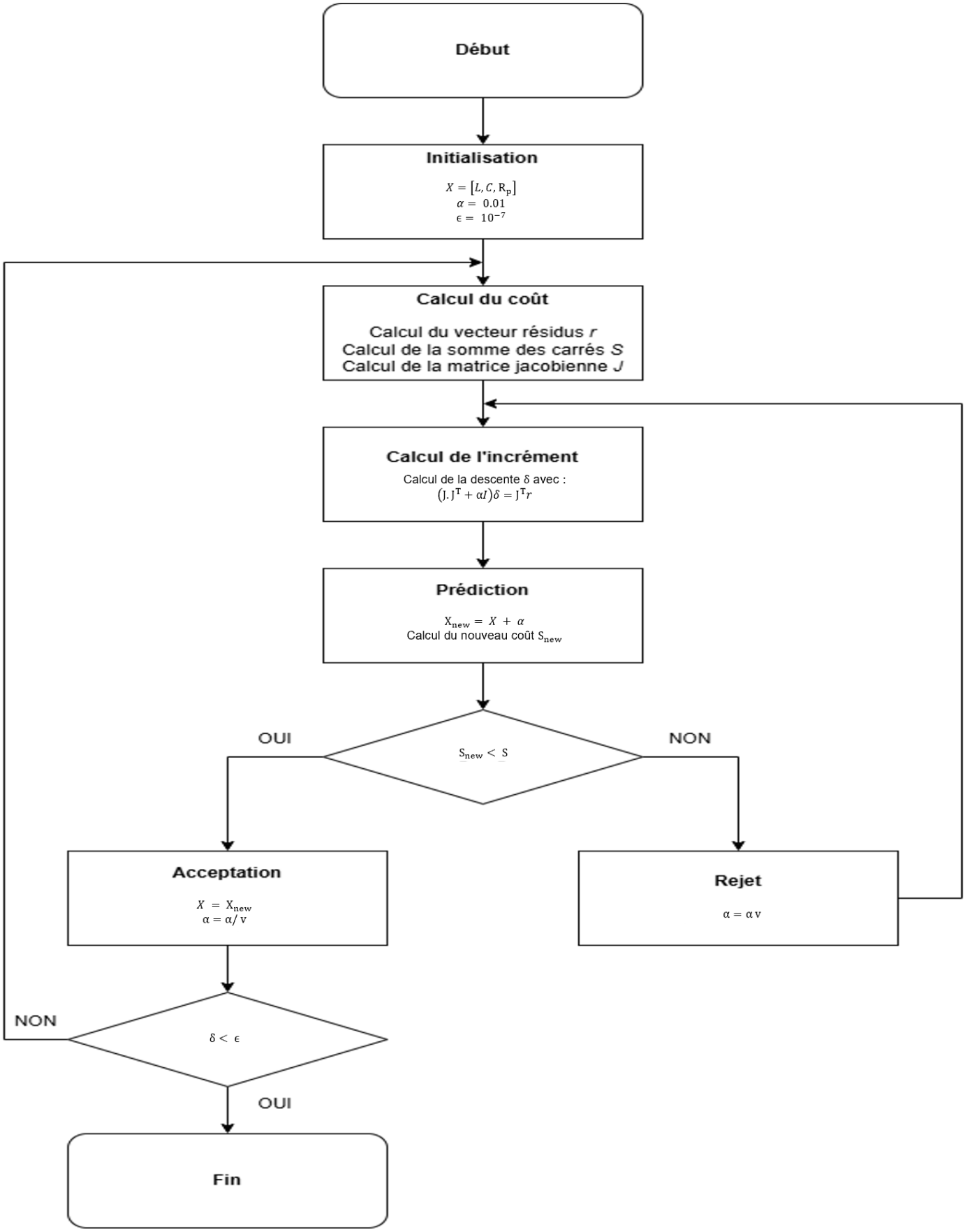


FIGURE 2 – Schéma de principe de l'algorithme

```python
import numpy as np

def Levenberg_Marquardt(X_init:np.ndarray, rs:float, f:np.ndarray, Z_exp:np.ndarray, N:int, learning_rate:float):
    """Algorythme de Levenberg-Marquardt pour l'optimisation non-lineaire des parametres electriques du micro

    Args:
        X_init (np.ndarray): Vecteur initial des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
        rs (float): Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
        Z_exp (np.ndarray): Tableau Numpy des impedances experimentale releve en Ohm
        N (int): Nombre d'iterations de l'algorithme

    Returns:
        np.ndarray: Vecteur optimisé des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
    """
    X = np.array(X_init, dtype=float)
    alpha = learning_rate
    v = 3
    r = None
    eps = None

    for _ in range(N):
        r = residus(X, rs, f, Z_exp)
        eps = np.sum(r**2)

        # 1. Calcul de la Jacobienne classique
        J = Matrice_Jacobienne(X, rs, f)

        # 2. NORMALISATION (Adimensionnement)
        # On multiplie chaque colonne de J par la valeur actuelle du paramètre X correspondant.
        # Cela permet de calculer des dérivées relatives (sensibilité à une variation en %)
        J_norm = J * X

        # 3. Calcul de la matrice Hessienne et du gradient normalisés
        H = J_norm.T @ J_norm
        g = J_norm.T @ r

        # Amortissement de Marquardt
        H_lm = H + alpha * np.diag(np.diag(H) + 1e-8)

        try:
            # delta_rel contient maintenant les variations relatives (ex: 0.05 = +5%)
            delta_rel = np.linalg.solve(H_lm, g)
        except np.linalg.LinAlgError:
            break

        # 4. Mise à jour des paramètres (X_new = X + X * delta_rel)
        X_new = X * (1 + delta_rel)

        # Contrainte physique : paramètres toujours positifs
        X_new = np.maximum(X_new, [1e-2, 1e-2, 1e-2])

        eps_new = np.sum(residus(X_new, rs, f, Z_exp)**2)

        if eps_new < eps:
            alpha /= v
            X = X_new
            # Condition d'arrêt sur la norme des variations relatives
            if np.linalg.norm(delta_rel) < 1e-7:
                break
        else:
            alpha *= v

    return X, eps_new
```

1.2.5 Calcul du diagramme de Bode d'un micro chargé

```python
import numpy as np

def calcul_bode(X:np.ndarray, rs:float, f:np.ndarray, Ramp:float=1e6)-> tuple[np.ndarray, np.ndarray]:
    """Calcul la fonction de transfert en gain et phase d'un micro de guitare en fonction de ses parametres internes
    Args:
        X (np.ndarray): Vecteur optimisé des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
        rs (float): Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
        Ramp (float, optional): Resistance d'entree de l'amplificateur. Defaults to 1e6.
    Returns:
        tuple[np.ndarray, np.ndarray]: Deux tableaux Numpy contenant respectivement le gain (en dB) et la phase (en degres) de la fonction de transfert du micro
    """
    w = 2 * np.pi * f
    L, C, Rp = X[0], X[1] * 1e-9, X[2] * 1e3

    # Impedance serie
    Z_ser = rs + 1j * L * w

    # Impedance parallèle
    Y_para = (1 / Rp) + (1 / Ramp) + (1j * C * w)
    Z_para = 1 / Y_para

    # Fonction de transfert
    H = Z_para / (Z_ser + Z_para)

    gain = 20 * np.log10(np.abs(H))
    phase = np.angle(H, deg=True)
    return gain, phase
```

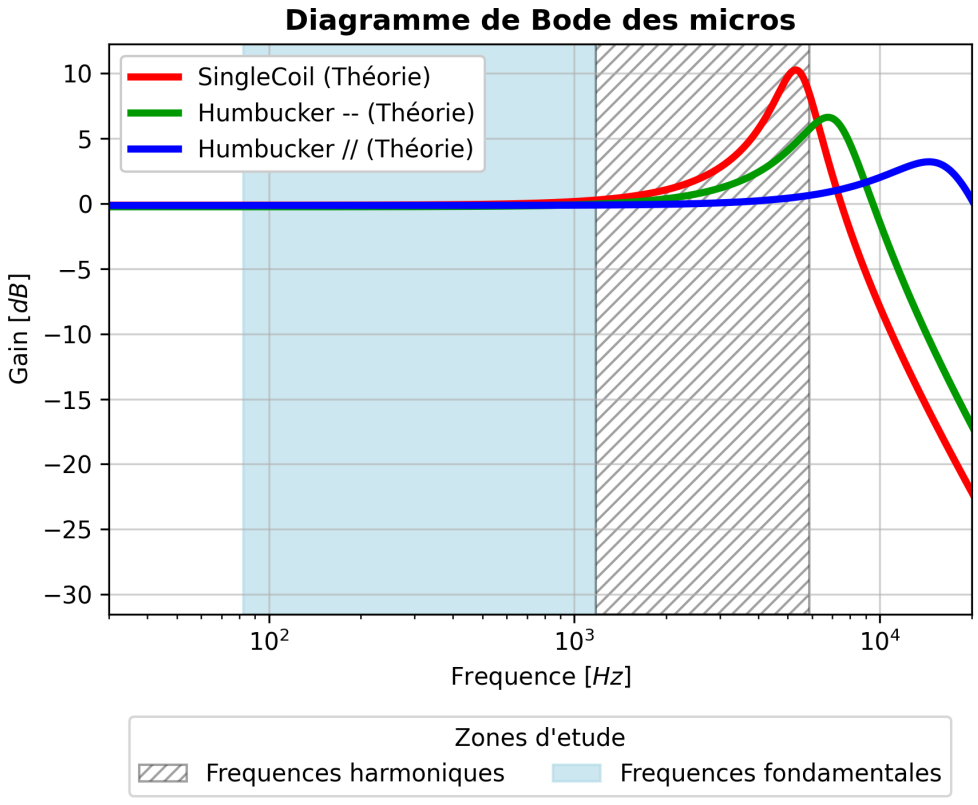


FIGURE 3 – Diagramme de Bode des micros

1.3 Mise en évidence du filtrage spatial

```python
import numpy as np
from scipy.signal import find_peaks

def extraire_pics_separes(frequences, amplitudes, prominence=0.05, distance=10):
    """Extrait les pics significatifs d'un spectre de Fourier en utilisant la proeminence."""
    indices, _ = find_peaks(amplitudes, prominence=prominence, distance=distance)
    liste_frequences = frequences[indices].tolist()
    liste_amplitudes = amplitudes[indices].tolist()
    return np.array(liste_frequences), np.round(liste_amplitudes, 3)

kw_peigne = {'color': "#000000", 'label': 'Peigne spatial', 'linewidth': 3}
kw_att = {'color': "#888888", 'label': 'Gain relatif', 'linewidth': 1}

L_freqs, L_pic_s_exp = mod.extraire_pics_separes(freqs_s_exp, fft_s_exp, prominence=0.04,
    distance=5)
L_freqs, L_pic_hs_exp = mod.extraire_pics_separes(freqs_hs_exp, fft_hs_exp, prominence=0.03,
    distance=5)

freqs = np.linspace(30, 20000, 1000)
signal_hs, peigne_hs, signal_sc = mod.attenuation_theorique_db(freqs, x_hs1, x_hs2)
signal_hp, peigne_hp, signal_sc = mod.attenuation_theorique_db(freqs, x_hp1, x_hp2)

Attenuation_th = 20*np.log10(np.abs(signal_hs / (signal_sc + 1e-6)) + 1e-6)
Attenuation_exp = 20*np.log10(np.abs(L_pic_hs_exp / (L_pic_s_exp + 1e-6)) + 1e-6)
mod.save_plot(PATH_exp + "Filtrage_spatial.png",
        (freqs, Attenuation_th, kw_att),
        (L_freqs, Attenuation_exp, kw_Hs_exp),
        (freqs, 20*np.log10(np.abs(peigne_hs)+1e-6), kw_peigne),
        title=f"Filtrage spatial du micro Humbucker série",
        xlabel="Frequence [$Hz$]", ylabel="Gain [dB]", xlim=(30, 20000), vspan=True,
        log=True)
```

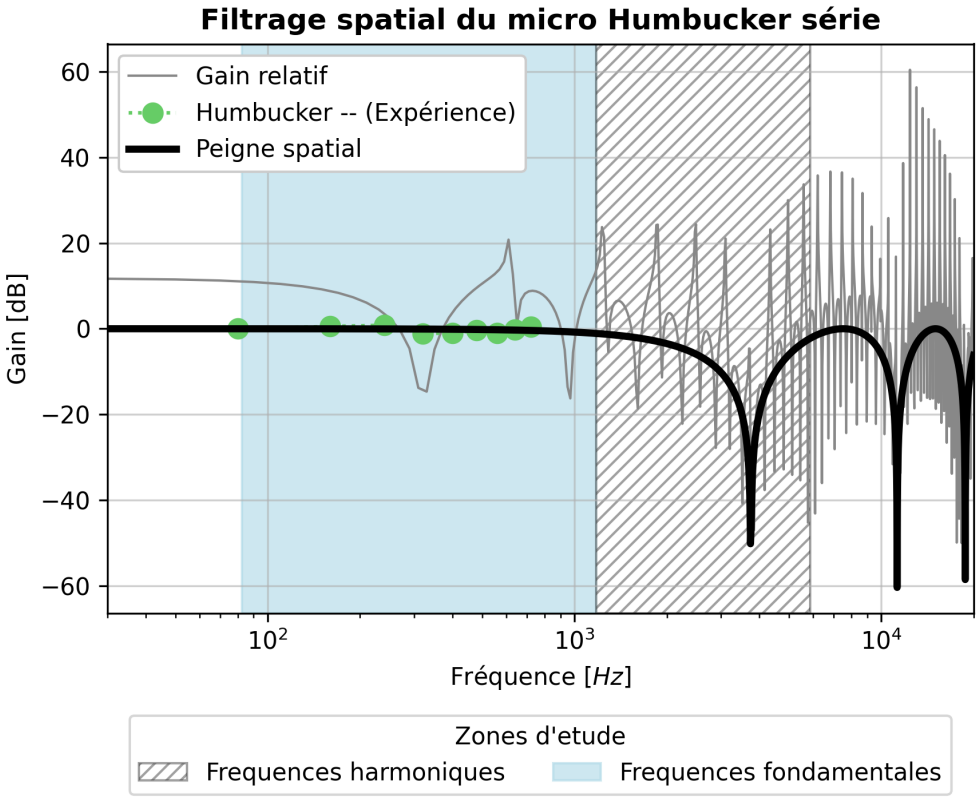


FIGURE 4 – Filtrage spatial du Humbucker série

2 Traitement des données expérimentales

2.1 Filtrage des données (Passe-bas numérique)

```python
import numpy as np

def lire_fichier(PATH):
    List_temps = []
    List_sig = []
    file_s2 = open(PATH)
    L_lignes = file_s2.readlines()

    for ligne in L_lignes[2:]:
        ligne = ligne.strip().split(',')
        ligne = [x.replace('\n', '') for x in ligne]
        ligne = [x.replace('E', 'e') for x in ligne]
        List_temps.append(float(ligne[0]))
        List_sig.append(float(ligne[1]))

    return List_temps, List_sig

def write_file(PATH, List_temps, List_sig):
    with open(PATH, 'w') as file:
        file.write('Time;Signal\n')
        for t, s in zip(List_temps, List_sig):
            file.write(f"{t}\t{s}\n")

def filtre_passe_bas_1er_ordre(List_temps, List_sig, tau=0.001):
    """Filtre passe-bas du 1er ordre (IIR)."""
    sig_array = np.array(List_sig)

    if tau < 0:
        tau = 0.001

    if len(List_temps) > 1:
        dt = (List_temps[-1] - List_temps[0]) / (len(List_temps) - 1)
    else:
        dt = 1.0

    alpha = dt / (tau + dt)

    filtered_sig = np.zeros_like(sig_array)
    filtered_sig[0] = sig_array[0]
    for i in range(1, len(sig_array)):
        filtered_sig[i] = alpha * sig_array[i] + (1 - alpha) * filtered_sig[i - 1]

    return filtered_sig
```

2.2 Amplitude d'un signal périodique

```python
def get_amplitude(PATH):
    List_temps = []
    List_sig = []
    file = open(PATH)

    L_lignes = file.readlines()
    for ligne in L_lignes[1:70000]:
        ligne = ligne.strip().split('\t')
        ligne = [x.replace('\n', '') for x in ligne]
        ligne = [x.replace(',', '.') for x in ligne]
        List_temps.append(float(ligne[0]))
        List_sig.append(float(ligne[1]))

    amplitude = max(List_sig) - min(List_sig)
    return amplitude
```

3 Données et valeurs numériques

3.1 Caractéristiques électriques et mécaniques

Notation	Valeur	Unité	Description
μ_0	$4\pi \cdot 10^{-7}$	$\mathrm{H{\cdot}m^{-1}}$	Perméabilité magnétique du vide
μ_r	2	–	Perméabilité relative de l'Alnico 5
R_{s_S}	$5,26 \cdot 10^3$	Ω	Résistance interne du micro SingleCoil
L_S	2,37	H	Inductance du micro SingleCoil
C_S	$0,37 \cdot 10^{-9}$	F	Capacité interne du micro SingleCoil
R_{p_S}	$492,3 \cdot 10^3$	Ω	Résistance des pertes par courant de Foucault
$R_{s_{\mathrm{Hp}}}$	$4,52 \cdot 10^3$	Ω	Résistance interne du micro Humbucker parallèle
L_{Hp}	1,72	H	Inductance du micro Humbucker parallèle
C_{Hp}	$0,05 \cdot 10^{-9}$	F	Capacité interne du micro Humbucker parallèle
$R_{p_{\mathrm{Hp}}}$	$343,0 \cdot 10^3$	Ω	Résistance des pertes par courant de Foucault
$R_{s_{\mathrm{Hs}}}$	$15,10 \cdot 10^3$	Ω	Résistance interne du micro Humbucker série
L_{Hs}	4,60	H	Inductance du micro Humbucker série
C_{Hs}	$0,11 \cdot 10^{-9}$	F	Capacité interne du micro Humbucker série
$R_{p_{\mathrm{Hs}}}$	$1,05 \cdot 10^6$	Ω	Résistance des pertes par courant de Foucault
x_S	0,105	m	Position du micro SingleCoil
$x_{\mathrm{Hp1}}, x_{\mathrm{Hp2}}$	0,147 - 0,165	m	Position des bobinages du Humbucker parallèle
$x_{\mathrm{Hs1}}, x_{\mathrm{Hs2}}$	0,202 - 0,220	m	Position des bobinages du Humbucker série
Δ	0,018	m	Entraxe entre les deux bobines du Humbucker
z_0	0,004	m	Distance corde micro au repos
L	0,85	m	Longueur de la corde vibrante
x_0	0,20	m	Position de l'origine de la perturbation
A_0	0,005 (ODG)	m	Amplitude initiale de la perturbation
F	80	Hz	Fréquence fondamentale de vibration de la corde

3.2 Caractéristiques magnétiques

Notation	Valeur	Unité	Description
χ	100 (ODG)	–	Susceptibilité magnétique de la corde
V	$1 \cdot 10^{-8}$ (ODG)	$\mathrm{m^3}$	Volume de la portion de la corde active
F_B	50	Hz	Fréquence du champ magnétique parasite
α	1 (ODG)	$\mathrm{m^{-1}}$	Gradient du champ magnétique parasite
k_s	2,02	$V.mT^{-1}$	Sensibilité théorique du SingleCoil
k_{Hp}	0,02	$V.mT^{-1}$	Sensibilité théorique du Humbucker parallèle
k_{Hs}	0,04	$V.mT^{-1}$	Sensibilité théorique du Humbucker série
k_{Sexp}	2,05	$V.mT^{-1}$	Sensibilité expérimentale du SingleCoil
k_{Hpexp}	1,37	$V.mT^{-1}$	Sensibilité expérimentale du Humbucker parallèle
k_{Hsexp}	0,08	$V.mT^{-1}$	Sensibilité expérimentale du Humbucker série
CMRR_{Hp}	39,3	dB	CMRR théorique du Humbucker parallèle
CMRR_{Hs}	35,0	dB	CMRR théorique du Humbucker série
$\mathrm{CMRR}_{\mathrm{Hpexp}}$	3,5	dB	CMRR expérimentale du Humbucker parallèle
$\mathrm{CMRR}_{\mathrm{Hsexp}}$	28,7	dB	CMRR expérimentale du Humbucker série

Références

[1] WiseGuy Research Consultants, *Global Electric Guitar Pickup Market Research Report 2024*, Pune, India, 2024.

[2] Matthew W. Hill, *The hidden history of the electric guitar", in Quand la guitare [s']électrise !*, Sorbonne Université Presses, pp. 33-61, 2022.

[3] N. G. Horton T. R. Moore, *Modeling the magnetic pickup of an electric guitar* , American Journal of Physics, vol. 77, no. 2, pp. 144–150, 2009, doi : 10.1119/1.2990663.

[4] S. W. Duncan, *Interview With Seth Lover*, [En ligne] Disponible sur : www.seymourduncan.com/blog/latest-updates/seymour-w-duncans-interview-with-seth-lover

[5] Dan Carson, *Investigation of the Electromagnetic Properties of Electric Guitar Pickups*, University of Illinois at Urbana-Champaign, Physics Senior Thesis, 2009.

[6] B. d'Andréa-Novel, B. Fabre, et P. Jouvelot, *Acoustique Informatique MusiquE : Outils scientifiques pour la musique*, Paris, France : Presses des Mines, 2010, ch. 4